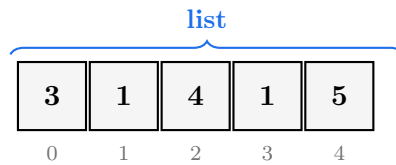


{ Estructuras de Datos }

Resumen de Contenidos

IIC2233 — Programación Avanzada

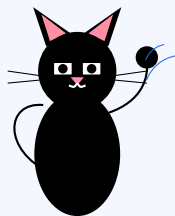
Resumen elaborado por el equipo de ayudantía, en base al material oficial del curso



Contents

1	Introducción	2
1.1	Clasificación de las estructuras de datos	2
2	Tuplas	3
2.1	Inmutabilidad y hasheabilidad	3
2.2	Desempaquetado de elementos	4
2.3	<i>Slicing</i> de tuplas	4
2.4	<i>Named tuples</i>	4
3	Stacks (Pilas)	5
3.1	Operaciones sobre stacks	5
3.2	Implementación en Python	5
3.3	Ejemplos de uso	6
4	Colas (Queues)	6
4.1	Operaciones sobre colas	7
4.2	<i>Deque</i> : colas de doble extremo	7
4.3	<code>list</code> vs. <code>deque</code> : comparación de eficiencia	7
4.4	Ejemplo de uso: palíndromos	8
5	Diccionarios	9
5.1	Diccionarios en Python	9
5.2	Eficiencia y llaves permitidas	9
5.3	Métodos útiles	10
5.4	Diccionarios por comprensión	10
5.5	Aplicación: conteo de frecuencias	10
5.6	<code>Defaultdict</code>	11

6	Sets	11
6.1	Operaciones básicas	11
6.2	Eficiencia: sets vs. listas	12
6.3	Operaciones de conjuntos	12
6.4	Comparar conjuntos	12
6.5	Aplicación: eliminar duplicados	12
7	args y kwargs	12
7.1	Argumentos posicionales y por palabra clave	13
7.2	Desempaquetado en llamadas: * y **	13
7.3	Cantidad variable de parámetros: *args y **kwargs	13
8	Combinando estructuras	14
8.1	Sets y diccionarios con tuplas	14
8.2	Diccionarios con sets	15
9	Ejercicios de práctica	15



Este es Felipín. Felipín es un gato nerd al que no le gustan las estructuras de datos: para **todo** usa list, sin importar si necesita buscar duplicados, mapear llaves a valores, o sacar elementos por el inicio miles de veces.

No seas como Felipín. Elegir la estructura correcta hace que tu código sea más simple, más claro y muchísimo más rápido.

1 Introducción

En los modelos tradicionales de programación estamos permanentemente manipulando datos. Para facilitar esta manipulación existen construcciones que permiten **agrupar y operar** sobre conjuntos de datos de forma eficiente: las **estructuras de datos**.

Una estructura de datos consiste en una manera de agrupar información relacionada, junto con un conjunto de **operaciones para acceder y modificar** esos datos de manera eficiente. A diferencia de tipos simples como int o float, las estructuras de datos involucran un mayor nivel de *abstracción*.

★ Iterar / Iterable

Iterar es recorrer una estructura, y algo es **iterable** cuando puede ser recorrido. Todas las estructuras que veremos en este resumen son iterables.

La elección de *qué* estructura usar depende tanto del contexto de la aplicación como de la **eficiencia** esperada para las operaciones que se necesitan.

1.1 Clasificación de las estructuras de datos

Clasificaremos las estructuras según sus propiedades en dos grandes categorías:

■ Secuenciales vs. no secuenciales

- **Secuenciales:** basadas en un **ordenamiento** de los elementos, según cómo fueron ingresados. Permiten recorrerse en un orden determinado y acceder al n -ésimo elemento. Incluyen los *built-ins* tuple, list, str, y las estructuras de *stack* y *cola*.
- **No secuenciales:** almacenan los datos **sin un orden fijo** de acceso. No garantizan en qué orden se entregan sus elementos, pero a cambio ofrecen métodos muy **eficientes de búsqueda**. Los diccionarios y los *sets* son las dos estructuras no secuenciales de este resumen (también existen árboles, grafos y *heaps*).

2 Tuplas

Las **tuplas** (tuple) se utilizan para manejar datos de forma **ordenada** e **inmutable**: una vez creada, no se pueden cambiar sus valores. Se accede a sus elementos mediante índices correlativos al orden en que fueron agregados. Suelen ser **heterogéneas**, es decir, pueden contener objetos de distintos tipos, incluyendo otras tuplas.

```
# Tupla vacia
a = tuple()

# Declaracion explicita entre parentesis
b = (0, 1, 2)

# Tupla de tamano 1: se necesita la coma final
c = (0, )

# Los parentesis son opcionales al crearla
d = 0, 'uno'

print(type(a), a) # <class 'tuple'> ()
print(type(b), b, b[0], b[1]) # <class 'tuple'> (0, 1, 2) 0 1
print(type(c), c) # <class 'tuple'> (0,)
print(type(d), d, d[0], d[1]) # <class 'tuple'> (0, 'uno') 0 uno
```

2.1 Inmutabilidad y hasheabilidad

Al ser inmutables, **no es posible agregar, eliminar, ni reemplazar** elementos de una tupla una vez creada; intentarlo produce un `TypeError`.

```
a = ('Chile', 2, 4.15, 'Agosto')
a[2] = 'semestre'
# TypeError: 'tuple' object does not support item assignment
```

Un gran beneficio de las estructuras inmutables es que son **hasheables**. Un *hash* es un número que Python calcula a partir del contenido de un objeto, y permite comparar dos objetos de forma eficiente sin recorrerlos elemento por elemento. Dos tuplas con el mismo contenido comparten el mismo *hash*.

! Ojo con el contenido

Una tupla es hasheable **solo si todos sus elementos también lo son**. Una tupla que contiene una list en su interior **no** es hasheable, porque las listas son mutables y list no implementa `__hash__`. La solución es usar tuplas anidadas en vez de listas anidadas cuando se necesite hashear.

Es importante notar que **sí** se puede modificar un valor *interno* de un objeto mutable que está guardado dentro de una tupla (por ejemplo, una lista en la posición 3); en ese caso no se está modificando la tupla en sí, sino el objeto al que ella apunta.

2.2 Desempaquetado de elementos

Las tuplas pueden **desempaquetarse** en variables individuales. Esto es lo que ocurre, por ejemplo, cuando una función retorna “varios valores”: en realidad los empaqueta en una sola tupla.

```
def calcular_geometria(a: float, b: float):
    area = a * b
    perimetro = (2 * a) + (2 * b)
    return (area, perimetro, a / 2, b / 2)

data = calcular_geometria(20.0, 10.0) # (200.0, 60.0, 10.0, 5.0)

# Desempaquetado directo en variables independientes
area, per, mpa, mpb = calcular_geometria(20.0, 10.0)
```

Además del desempaquetado simple, Python ofrece el **operador de desempaquetado** *, que permite capturar “el resto” de los elementos en una lista mientras se extraen el primero y el último (o cualquier combinación fija de posiciones):

```
datos = ('inicio', 39, 38, 12, 80, 51, 10, 20, 95, 'fin')

primero, *relleno, ultimo = datos
# primero = 'inicio'
# relleno = [39, 38, 12, 80, 51, 10, 20, 95] (queda como LISTA)
# ultimo = 'fin'
```

! No confundir

El operador * tiene dos usos distintos según el contexto: al **crear** una tupla/lista, *expande* un iterable dentro de ella; al **desempaquetar** en asignación, *recolecta* varios elementos en una lista.

2.3 Slicing de tuplas

Al igual que con listas, se pueden tomar secciones de una tupla usando la notación de *slicing*: `tupla[inicio:fn:passo]`.

```
data = (400, 20, 1, 4, 10, 11, 12, 500)

data[1:3] # (20, 1) -> entre indices 1 y 3 (sin incluir el 3)
data[3:] # (4, 10, 11, 12, 500) -> desde el indice 3 en adelante
data[:5] # (400, 20, 1, 4, 10) -> hasta el indice 5 (sin incluirlo)
data[2::2] # (1, 10, 12) -> desde el indice 2, de a dos en dos
data[1:6:2] # (20, 4, 11) -> entre 1 y 6, de a dos en dos
data[::-1] # (500, 12, 11, 10, 4, 1, 20, 400) -> invertida
```

2.4 Named tuples

Las **named tuples** permiten asignar **nombres semánticos** a cada posición de una tupla, mejorando la claridad y explicitud del código. Se importan desde el módulo `collections`.

```
from collections import namedtuple

# Nombre del tipo y nombres de los campos (string o lista de strings)
```

```
ChileanRegister = namedtuple('ChileanRegister_type', ['RUT', 'name', 'age'])

c1 = ChileanRegister('13427974-5', 'Christian', 20)
print(c1.name) # Christian (acceso por atributo, no solo por indice)
```

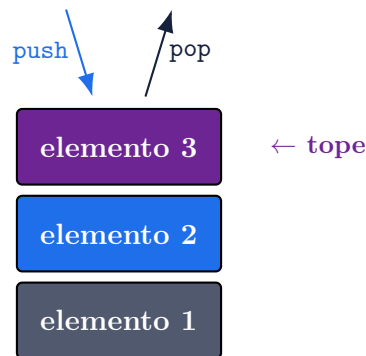
Las *named tuples* son igual de **inmutables** que las tuplas normales (`c1.name = 'x'` lanza `AttributeError`). Para “modificarlas” existe el método `_replace`, que retorna una **copia nueva** con los valores indicados reemplazados:

```
c1_mod = c1._replace(name="Antonio", age=31)
# ChileanRegister_type(RUT='13427974-5', name='Antonio', age=31)
```

Un uso muy común es representar **coordenadas**, ya que permite acceder a los valores de forma más ordenada y legible que con una tupla común (por ejemplo, `p.row` en vez de `p[0]`).

3 Stacks (Pilas)

Un *stack* (o pila) es una estructura que funciona como una pila de objetos, uno arriba del otro: pensemos en una pila de platos por lavar. Cada plato nuevo se coloca arriba, y cada plato que se saca para lavar es siempre el de más arriba, es decir, el **último** que se agregó.



LIFO — Last In, First Out: el último en entrar es el primero en salir

3.1 Operaciones sobre stacks

■ Operaciones básicas

- **Push:** agrega un elemento al **tope** del stack.
- **Pop:** elimina y retorna el elemento del tope (siempre el último agregado).
- **Peek:** muestra el elemento del tope **sin** sacarlo.

También es posible consultar la cantidad de elementos, o si el stack está vacío.

El stack es una estructura de tipo **LIFO** (*Last In, First Out*): lo último que entra es lo primero en salir. Por lo tanto, para sacar el primer elemento agregado hay que sacar antes todos los demás.

3.2 Implementación en Python

En Python, un *stack* se representa naturalmente con una `list`:

Operación	Código Python	Descripción
Crear <i>stack</i>	<code>stack = []</code>	Crea un stack vacío
<i>Push</i>	<code>stack.append(x)</code>	Agrega x al tope del stack
<i>Pop</i>	<code>stack.pop()</code>	Retorna y extrae el elemento del tope
<i>Peek</i>	<code>stack[-1]</code>	Retorna el tope sin extraerlo
<i>Length</i>	<code>len(stack)</code>	Cantidad de elementos
<i>Is empty</i>	<code>len(stack) == 0</code>	True si está vacío

```
stack = []
stack.append(1)
stack.append(2)
stack.append(3) # stack: [1, 2, 3]

elemento = stack.pop() # elemento = 3, stack: [1, 2]
tope = stack[-1] # tope = 2 (el stack queda intacto)
```

3.3 Ejemplos de uso

Un caso real de uso de *stacks* es el botón “**atrás**” de un navegador: cada URL visitada se apila, y al retroceder se hace *pop* de la última.

```
class Navegador:
    def __init__(self, current_url='https://www.google.com'):
        self.__urls_stack = []
        self.__current_url = current_url

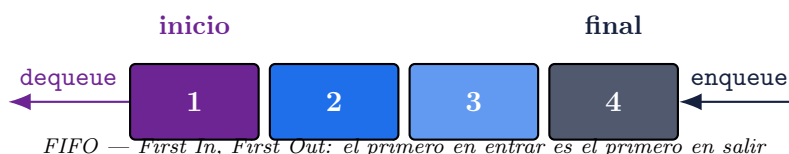
    def ir(self, url):
        self.__urls_stack.append(self.__current_url)
        self.__current_url = url

    def volver(self):
        self.__current_url = self.__urls_stack.pop()
```

Otro uso típico es **invertir una secuencia**: se apilan todos los elementos y luego se van sacando, obteniéndolos en orden inverso al de llegada (por ejemplo, invertir las líneas de un texto).

4 Colas (Queues)

Una **cola** es una estructura secuencial que mantiene los objetos ordenados según su **orden de llegada**, igual que una fila para pagar: quien llega se pone al final, y se atiende primero a quien lleva más tiempo esperando. Por esto es una estructura **FIFO** (*First In, First Out*).



4.1 Operaciones sobre colas

■ Operaciones básicas

- **Enqueue:** agrega un elemento al **final** de la cola.
- **Dequeue:** saca el elemento del **inicio** (el que lleva más tiempo esperando).
- **Peek:** muestra el primer elemento sin sacarlo.

! ¿Por qué no usar una lista?

Con una *list*, `append` (*enqueue*) es directo y eficiente. Pero `pop(0)` (*dequeue*) **no lo es**: una lista guarda sus elementos en un bloque contiguo de memoria, así que al sacar el primero, Python debe **desplazar una posición a la izquierda** todos los $N - 1$ elementos restantes. Esto hace que `pop(0)` tome un tiempo **proporcional al tamaño** de la lista, lo cual es ineficiente para colas grandes.

4.2 Deque: colas de doble extremo

El **deque** (*double-ended queue*, del módulo `collections`) permite **agregar y sacar elementos desde ambos extremos** con **costo constante** por operación, sin importar cuántos elementos tenga.

Operación	Código Python	Descripción
Crear <i>deque</i>	<code>deque()</code>	Crea un deque vacío
<i>Add last</i>	<code>d.append(x)</code>	Agrega x al final
<i>Add first</i>	<code>d.appendleft(x)</code>	Agrega x al inicio
<i>Delete first</i>	<code>d.popleft()</code>	Extrae y retorna el primer elemento
<i>Delete last</i>	<code>d.pop()</code>	Extrae y retorna el último elemento
<i>First / Last</i>	<code>d[0] / d[-1]</code>	Accede sin extraer
<i>length</i>	<code>len(d)</code>	Cantidad de elementos

```
from collections import deque

cola = deque()
cola.append(1); cola.append(2); cola.append(3) # deque([1, 2, 3])

elemento = cola.popleft() # elemento = 1, cola: deque([2, 3])
primero = cola[0] # primero = 2 (sin extraer)
```

! Acceso indexado en deque

`deque[i]` funciona, pero **no es eficiente** como en una lista: para llegar a la posición *i* hay que recorrer todos los elementos anteriores.

4.3 list vs. deque: comparación de eficiencia

Para entender por qué deque conviene más que list al implementar una cola, hay que mirar **cómo organiza cada estructura sus elementos en memoria**.

■ ¿Por qué existe esta diferencia de rendimiento?

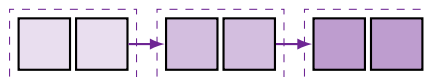
- Una list guarda sus elementos en un **bloque contiguo** de memoria, uno justo al lado del otro. Gracias a eso, la posición de cualquier elemento se calcula con una fórmula directa ($\text{dirección_inicio} + i$), por lo que **acceder por índice es inmediato**, sin importar el tamaño de la lista. Pero insertar o sacar el **primer** elemento obliga a **recorrer y desplazar** todos los demás una posición hacia el lado contrario, lo que toma un tiempo **proporcional al tamaño** de la lista.
- Un deque se organiza como una **cadena de bloques enlazados** entre sí (parecido a una lista enlazada de bloques pequeños, en vez de un único bloque gigante). Agregar o sacar un elemento en **cualquiera de los dos extremos** solo requiere modificar el bloque de esa punta, **sin tocar el resto** de la estructura: por eso `appendleft`, `popleft`, `append` y `pop` son **inmediatos**, sin importar cuántos elementos tenga. En cambio, para llegar a un elemento del **medio** no existe una fórmula directa: hay que **recorrer bloque por bloque** desde el extremo más cercano, por lo que el acceso por índice toma un tiempo proporcional a qué tan lejos esté ese elemento.

list: bloque contiguo en memoria



índice → dirección directa: acceso inmediato, pero `pop(0)` desplaza todo

deque: bloques enlazados



`appendleft`: inmediato

`append`: inmediato

Un experimento con 10^7 elementos confirma esta diferencia en la práctica:

■ Salida

Buscar el elemento del medio: en el **deque** es mucho más lento que en la **list**, porque hay que recorrer bloque por bloque en vez de calcular la posición directamente.

Sacar 1000 elementos del inicio (`popleft` vs. `pop(0)`): en el **deque** es miles de veces más rápido que en la **list**, porque no requiere desplazar el resto de los elementos cada vez.

★ Idea central

Acceder por índice (especialmente al medio) es más rápido en **list**, porque sus elementos están en un bloque contiguo. **Agregar/sacar por los extremos** es más rápido en **deque**, porque no necesita desplazar el resto de los elementos. Por eso **deque** es la estructura recomendada para implementar colas y stacks: **generaliza** ambas operaciones con la misma eficiencia.

4.4 Ejemplo de uso: palíndromos

Un deque es ideal para comparar los extremos de una palabra de forma recursiva hasta que quede una sola letra (o ninguna), verificando si es palíndroma:

```
from collections import deque

def es_palindromo_rec(palabra):
    if len(palabra) <= 1:
        return True
    return palabra.popleft() == palabra.pop() and es_palindromo_rec(palabra)
```



```
def es_palindromo(palabra):
    return es_palindromo_rec(deque(palabra))
```

? ¿Hay una forma más directa?

Sí: en Python basta con comparar `palabra == palabra[::-1]`. El ejemplo anterior sirve para practicar el uso de `deque` y recursión.

5 Diccionarios

Un **diccionario** es una estructura de datos **no secuencial** y **mutable** que asocia pares **llave-valor** (*key-value*). Se consulta por una llave y se obtiene su valor asociado; llave y valor pueden ser de tipos distintos. También se le llama estructura de “mapeo” (*mapping*), y en otros lenguajes se conoce como *hash table* o *hash map*.

5.1 Diccionarios en Python

En Python están implementados por la clase `dict`, y se escriben con llaves `{}`, separando cada par con `:`.

```
monedas = {
    "Chile": "Peso",
    "Peru": "Soles",
    "Espana": "Euro",
    "Brasil": "Real",
}

monedas["Chile"] # 'Peso'
monedas.get('Peru', 0) # 'Soles'
monedas.get('Argentina', 'Sin moneda') # 'Sin moneda' (no lanza error)

monedas["Vaticano"] = "Lira" # crea la llave si no existia
monedas["Vaticano"] = "Euro" # la actualiza si ya existia

del monedas["Vaticano"] # elimina el par (KeyError si no existe)
'Chile' in monedas # True (in busca en las LLAVES, no en los valores)
```

! KeyError

Consultar una llave inexistente con `diccionario[llave]` lanza `KeyError`. El método `.get(llave, default)` evita ese error, retornando `default` (o `None` si no se especifica) cuando la llave no existe.

A diferencia de lenguajes como Java o C#, en Python **no es necesario** que todas las llaves sean del mismo tipo, ni que todos los valores lo sean.

5.2 Eficiencia y llaves permitidas

Acceder al valor de una llave (`dict[llave]`) es una operación **muy eficiente**: toma **tiempo constante**, sin importar el tamaño del diccionario. Esto se debe a que los diccionarios se implementan mediante una **tabla de hash**: una función matemática (*función de hash*) determina dónde guardar cada valor a partir de su llave.

! No toda llave es válida

Para ser una llave válida, un objeto debe ser **único** dentro del diccionario y **hasheable**, es decir:

1. Implementa `__hash__`, que retorna un entero fijo mientras el objeto exista.
2. Implementa `__eq__` para comparar igualdad.
3. Si dos objetos son iguales según `__eq__`, deben tener el mismo `__hash__`.

Los *built-ins* **inmutables** (int, str, tuple) son hasheables y sirven como llave. Los **mutables** (list, dict, set) **no** son hasheables y no pueden usarse como llave.

En cambio, **no hay restricciones** sobre los **valores**: pueden ser mutables o inmutables, e incluso otros diccionarios, funciones o clases.

5.3 Métodos útiles

■ Recorriendo un diccionario

- `dict.keys()`: todas las llaves.
- `dict.values()`: todos los valores.
- `dict.items()`: pares (llave, valor) como tuplas.

```
for llave, valor in monedas.items():
    print(f'La moneda de {llave} es {valor}')

# Iterar directo sobre el diccionario recorre solo las llaves
for llave in monedas:
    print(llave)
```

5.4 Diccionarios por comprensión

Al igual que con listas, es posible construir diccionarios usando **comprensión**, incluyendo condiciones dentro de ella:

```
from string import ascii_lowercase as letras

numero_por_letra = {letras[i].upper(): i + 1 for i in range(len(letras))}

numero_por_vocales = {
    letras[i].upper(): i + 1 for i in range(len(letras))
    if letras[i].upper() in "AEIOU"
}

# {'A': 1, 'E': 5, 'I': 9, 'O': 15, 'U': 21}
```

5.5 Aplicación: conteo de frecuencias

Una aplicación muy común de los diccionarios es **contar ocurrencias**, por ejemplo, de vocales en un texto:

```
vocales = dict()
for letra in "supercalifragilisticoespialidoso":
    if letra not in 'aeiou':
        continue
```

```
if letra not in vocales:
    vocales[letra] = 0
    vocales[letra] += 1
# {'u': 1, 'e': 2, 'a': 3, 'i': 6, 'o': 3}
```

5.6 Defaultdict

Un **defaultdict** (del módulo `collections`) asigna un **valor por defecto** a cualquier llave nueva, evitando tener que preguntar si la llave ya existe. Recibe una función *sin parámetros* que retorna el valor por defecto.

```
from collections import defaultdict

vocales = defaultdict(int) # int() = 0 por defecto
for letra in "supercalifragilisticoespialidoso":
    if letra in 'aeiou':
        vocales[letra] += 1 # ya no es necesario chequear si existe
```

★ También funciona con funciones propias

Se le puede pasar cualquier *callable* sin argumentos, por ejemplo una función que genere un número aleatorio o una lista vacía (`defaultdict(list)`) como valor por defecto.

6 Sets

Los **sets** (conjuntos) son contenedores **mutables**, **no ordenados**, que **no repiten elementos** y se comportan de forma similar a los conjuntos matemáticos. Pueden contener cualquier objeto **hasheable** (los mismos tipos válidos como llave de diccionario), ya que también se implementan mediante tablas de hash.

```
vacio = set() # set vacio -- OJO: {} crea un dict, no un set
artistas = {"Sting", "Mon Laferte", "Sting"} # {'Sting', 'Mon Laferte'} (sin duplicados)
```

! {} no crea un set vacío

{ } crea un **diccionario** vacío, no un set. Para un set vacío hay que usar `set()` explícitamente.

También se permite la **comprensión de sets**, con la misma sintaxis que listas y diccionarios.

6.1 Operaciones básicas

Operación	Código Python	Descripción
Tamaño	<code>len(s)</code>	Cantidad de elementos
<i>Add</i>	<code>s.add(x)</code>	Agrega x (no ocurre nada si ya estaba)
<i>Remove</i>	<code>s.remove(x)</code>	Elimina x; error si no existe
<i>Discard</i>	<code>s.discard(x)</code>	Elimina x; sin error si no existe
Pertenencia	<code>x in s</code>	Verifica si x está en s (eficiente)
Iterar	<code>for x in s:</code>	Recorre cada elemento exactamente una vez, en orden no garantizado

! Sin acceso indexado

Los *sets* **no** soportan acceso por índice (`s[0]` lanza error), ya que no tienen orden definido.

6.2 Eficiencia: sets vs. listas

Verificar si un elemento **pertenece** a un *set* toma **tiempo constante**, sin importar cuántos elementos tenga. En una *list*, en cambio, hay que recorrerla completa en el peor caso, por lo que el tiempo de búsqueda **crece con el tamaño**. Un experimento con 10^8 elementos mostró que la búsqueda en una lista puede ser más de 50 veces más lenta que en un *set* equivalente.

6.3 Operaciones de conjuntos

Operación	Operador	Método	Descripción
Unión	<code>a b</code>	<code>a.union(b)</code>	Elementos de <i>a</i> o <i>b</i> (o ambos)
Intersección	<code>a & b</code>	<code>a.intersection(b)</code>	Elementos en ambos
Diferencia	<code>a - b</code>	<code>a.difference(b)</code>	En <i>a</i> pero no en <i>b</i> (importa el orden)
Dif. simétrica	<code>a ^ b</code>	<code>a.symmetric_difference(b)</code>	En <i>a</i> o <i>b</i> , pero no en ambos

Ninguna de estas operaciones modifica los *sets* originales; siempre retornan uno **nuevo**.

6.4 Comparar conjuntos

- `a <= b` o `a.issubset(b)`: *a* es **subconjunto** de *b* (todo elemento de *a* está en *b*).
- `a >= b` o `a.issuperset(b)`: *a* es **superconjunto** de *b*.
- `a == b`: son iguales si tienen exactamente los mismos elementos.

6.5 Aplicación: eliminar duplicados

Convertir una lista a *set* y de vuelta a *list* es la forma más directa de **eliminar elementos repetidos** (perdiendo el orden original, salvo que luego se ordene explícitamente).

```
lista = ['A', 'B', 'A', 'D', 'F', 'X', 'X', 'Z', 'Z', 'Y']
sin_duplicados = list(set(lista))
```

7 args y kwargs

Los términos *args* y *kwargs* provienen de *arguments* y *keyword arguments*. Son la forma en que Python permite especificar una **cantidad variable de argumentos o parámetros** al definir y llamar funciones.

? Argumentos vs. parámetros

No son sinónimos: los **parámetros** son los **nombres** que recibe una función, declarados en su **definición**. Los **argumentos** son los **valores** efectivos entregados al **llamar** la función. Una función tiene un único conjunto de parámetros, pero puede recibir muchos conjuntos distintos de argumentos.

7.1 Argumentos posicionales y por palabra clave

Por defecto, el **orden** de los argumentos determina a qué parámetro corresponde cada uno (**argumento posicional**). Pero como los parámetros tienen nombre, también se pueden entregar en cualquier orden usando su nombre como **palabra clave** (*keyword argument*):

```
def ejemplo(a, b, c):
    print(f'a: {a}, b: {b}, c: {c}')

ejemplo('hola', 'mundo', 42) # posicional
ejemplo(c=42, a='hola', b='mundo') # por palabra clave, en cualquier orden
```

! Reglas de uso conjunto

Se pueden mezclar ambos tipos en una misma llamada, pero:

- No puede haber un argumento **posicional después** de uno por palabra clave.
- No se puede volver a asignar por palabra clave un parámetro **ya asignado** por posición.

7.2 Desempaquetado en llamadas: * y **

- `func(*iterable)`: desempaqueta una lista o tupla (o cualquier iterable) y entrega sus elementos como argumentos **posicionales**.
- `func(**diccionario)`: desempaqueta un diccionario y entrega sus pares como argumentos **por palabra clave**.

```
datos = ['hola', 'mundo', 42]
opciones = {'a': 'hola', 'b': 'mundo', 'c': 42}

ejemplo(*datos) # equivalente a ejemplo('hola', 'mundo', 42)
ejemplo(**opciones) # equivalente a ejemplo(a='hola', b='mundo', c=42)
```

Ambos mecanismos pueden combinarse entre sí y con argumentos explícitos en la misma llamada, siempre respetando las reglas de orden ya mencionadas.

7.3 Cantidad variable de parámetros: *args y **kwargs

El mecanismo más simple para admitir una cantidad variable de argumentos son los **valores por defecto** (`def f(a, b="mundo", c=42):`), pero la cantidad de argumentos posibles sigue siendo **acotada**. Para permitir una cantidad **arbitraria**, se usan * y ** en la **declaración** de la función:

■ *args y **kwargs

- ***args**: recibe una cantidad arbitraria de argumentos **posicionales**, empaquetados en una **tupla** llamada args.
- ****kwargs**: recibe una cantidad arbitraria de argumentos **por palabra clave**, empaquetados en un **diccionario** llamado kwargs.

```
def func1(*args):
    print(args) # tupla, ej: (1, 2, 3, 4)

def func2(**kwargs):
    print(kwargs) # dict, ej: {'nombre': 'Pedro'}
```

```
def func3(a, b=3, *args, **kwargs):
    print(a, b, args, kwargs)

func3(1, 2, 3, 4, c=5, d=6)
# a: 1, b: 2, args: (3, 4), kwargs: {'c': 5, 'd': 6}
```

★ Los nombres son solo convención

El comportamiento especial lo dan los operadores `*` y `**`, **no** las palabras `args`/`kwargs`: se puede usar cualquier nombre de variable. Por convención casi siempre se usan estos nombres.

! Restricciones de declaración

- Solo puede haber **una** declaración con `*` y **una** con `**` por función, y `**` nunca puede ir antes que `*`.
- No se pueden declarar parámetros normales **después** de `**kwargs`.
- Sí se pueden declarar parámetros **entre** `*args` y `**kwargs`: esos parámetros solo podrán entregarse **por palabra clave** (nunca posicionalmente).

Con esto, una función general puede combinar todos los tipos de parámetros:

```
def funcion_general(arg1, arg2, *args, kwarg1, kwarg2, **kwargs):
    pass
# arg1, arg2 -> posicionales o por palabra clave
# args -> variable de posicionales
# kwarg1/2 -> SOLO por palabra clave (van despues de *args)
# kwargs -> variable por palabra clave
```

8 Combinando estructuras

Muchas veces el mayor provecho de las estructuras de datos se obtiene al **combinarlas**, aprovechando las propiedades de cada una.

8.1 Sets y diccionarios con tuplas

Como las **tuplas** son **inmutables y hashables**, pueden guardarse dentro de set o usarse como **llave** de un dict. Un caso típico es almacenar coordenadas:

```
class PuntosDeInteres:
    def __init__(self):
        self.visitados = set()

    def agregar_punto_de_interes(self, latitud, longitud):
        # La tupla (longitud, latitud) es hashable -> se puede
        # guardar en un set, y no se repetiran pares iguales
        self.visitados.add((longitud, latitud))
```

De forma similar, se puede usar un **diccionario** con tuplas como llave para además guardar **información asociada** a cada punto:

```
class PuntosDeInteresConInfo:
    def __init__(self):
        self.visitados_con_info = {}

    def agregar_punto_de_interes(self, latitud, longitud, info):
        self.visitados_con_info[(longitud, latitud)] = info
```

8.2 Diccionarios con sets

Otra combinación muy útil es un **diccionario cuyos valores son sets**. Un caso clásico es un catálogo de productos clasificados por *tags*, donde se necesita buscar productos que cumplan con varios tags a la vez:

```
from collections import defaultdict

class Catalogo:
    def __init__(self):
        # defaultdict(set): si el tag no existe, se crea un set vacío
        self.productos_por_tag = defaultdict(set)

    def agregar_tag_a_producto(self, producto, tag):
        self.productos_por_tag[tag].add(producto)

    def buscar_productos_con_tags(self, tags):
        productos = self.productos_por_tag[tags[0]]
        for tag in tags[1:]:
            # Interseccion de sets: solo quedan los productos
            # que cumplen con TODOS los tags pedidos
            productos = productos.intersection(self.productos_por_tag[tag])
        return productos
```

★ Por qué funciona bien

Usar `defaultdict(set)` evita revisar si el *tag* ya existe antes de agregar un producto, y usar `set` como valor permite calcular **intersecciones** de forma directa y eficiente para responder consultas con múltiples criterios, algo mucho más incómodo de hacer con listas.

9 Ejercicios de práctica

■ Ejercicio 1 — Máximo número de “balloon” (LeetCode 1189)



Dado un *string* `text`, se quieren usar los caracteres de `text` para formar la mayor cantidad posible de instancias de la palabra **“balloon”**. Cada carácter de `text` se puede usar **como máximo una vez**. Retorna el número máximo de instancias que se pueden formar.

Ejemplo 1:

■ Entrada / Salida

```
text = "nlaebolko"
Output: 1
```

Ejemplo 2:

■ Entrada / Salida

```
text = "loonbalxballpoon"
Output: 2
```

Ejemplo 3:

■ Entrada / Salida

```
text = "leetcode"
Output: 0
```

✓ Solución

```
from collections import defaultdict

class Solution:
    def maxNumberOfBalloons(self, text: str) -> int:
        tabla_hash = defaultdict(int)
        for letra in text:
            tabla_hash[letra] += 1

        return min(tabla_hash["b"],
                    tabla_hash["a"],
                    tabla_hash["l"] // 2,
                    tabla_hash["o"] // 2,
                    tabla_hash["n"]
                    )
```

Paso 1 — Contar las letras disponibles. Se recorre `text` y se cuenta cuántas veces aparece cada letra usando un `defaultdict(int)`, que evita tener que revisar si la llave ya existe antes de sumar (ver Sección 5, *Defaultdict*).

Paso 2 — Relacionar las letras con la palabra objetivo. La palabra “balloon” tiene 7 letras, pero solo 5 distintas: b, a, l, o, n. De estas, l y o aparecen **dos veces** cada una dentro de la palabra.

Paso 3 — Identificar el cuello de botella. Cada instancia completa de “balloon” consume 1 b, 1 a, 2 l, 2 o y 1 n del texto disponible. La cantidad máxima de instancias que se pueden formar está limitada por la letra que “alcanza para menos” repeticiones, una vez descontada su frecuencia dentro de la palabra (por eso l y o se dividen por 2: cada instancia gasta dos de cada una).

Paso 4 — Calcular el mínimo. Se retorna el mínimo entre `tabla_hash["b"]`, `tabla_hash["a"]`, `tabla_hash["l"] // 2`, `tabla_hash["o"] // 2` y `tabla_hash["n"]`. Ese mínimo corresponde exactamente a la letra que primero se agota, y por lo tanto, a la cantidad máxima de veces que se puede formar “balloon” completa sin que falte ningún carácter.

■ Ejercicio 2 — Transformación de rango de un arreglo (LeetCode 1331)



Dado un arreglo de enteros `arr`, reemplaza cada elemento por su **rango** (*rank*). El rango representa qué tan grande es el elemento, según las siguientes reglas:

- El rango es un entero que comienza en 1.
- Mientras más grande el elemento, más grande su rango. Si dos elementos son iguales, deben tener el **mismo** rango.
- El rango debe ser lo más pequeño posible.

Ejemplo 1:

■ Entrada / Salida

```
arr = [40, 10, 20, 30]
Output: [4, 1, 2, 3]
```

40 es el elemento más grande. 10 es el más pequeño. 20 es el segundo más pequeño. 30 es el tercero más pequeño.

Ejemplo 2:

■ Entrada / Salida

```
arr = [100, 100, 100]
Output: [1, 1, 1]
```

Elementos iguales comparten el mismo rango.

Ejemplo 3:

■ Entrada / Salida

```
arr = [37, 12, 28, 9, 100, 56, 80, 5, 12]
Output: [5, 3, 4, 2, 8, 6, 7, 1, 3]
```

✓ Solución

```
class Solution:
    def arrayRankTransform(self, arr: List[int]) -> List[int]:
        # Ordenamos los valores únicos
        arr_ordenado = sorted(set(arr))

        # Creamos un diccionario con el rank de cada valor
        rank = {num: i + 1 for i, num in enumerate(arr_ordenado)}

        # Retornamos el rank correspondiente a cada elemento
        # del arreglo original
        return [rank[num] for num in arr]
```

Paso 1 — Obtener los valores únicos y ordenarlos. `sorted(set(arr))` primero elimina los duplicados usando un `set` (ver Sección 6) — por eso los elementos iguales terminarán compartiendo rango — y luego los ordena de menor a mayor.

Paso 2 — Asignar un rango a cada valor único. Con una **comprensión de diccionario** (ver Sección 5), `{num: i + 1 for i, num in enumerate(arr_ordenado)}`, se recorre la lista ya ordenada y a cada valor se le asigna su posición (partiendo desde 1), que es exactamente su rango: el valor más pequeño queda con rango 1, el siguiente con rango 2, y así sucesivamente.

Paso 3 — Traducir el arreglo original usando el diccionario. Con otra comprensión, `[rank[num] for num in arr]`, se recorre `arr` en su **orden original** y se reemplaza cada número por el rango que le corresponde según el diccionario construido en el paso anterior. Como buscar una llave en un diccionario toma tiempo constante (ver Sección 5, *Eficiencia y llaves permitidas*), esta traducción es mucho más eficiente que calcular la posición de cada elemento manualmente.

Conclusión. El resultado conserva el **orden original** del arreglo de entrada, pero cada valor fue reemplazado por el rango que le corresponde dentro del conjunto de valores únicos y ordenados.

Fin del resumen.